A New Wave Function Collapse Algorithm using Genetic Algorithm and Simulated Annealing Metaheuristics for the Betweenness Problem

1st Daniel Han Qin
Monta Vista High School
Cupertino, United States of America
danielhqin1127@gmail.com

2nd David Perkins
Department of Computer Science
Colgate University
Hamilton, United States of America
dperkins@colgate.edu

***Abstract*—The Betweeness Problem is an important NP-hard problem in computer science and combinatorics. This paper focuses on a new algorithm, WFC-SA-GA, that approximates solutions to the optimization version of the Betweeness Problem, which is also known as the Maximum Betweeness Problem (MBP). This approach uses the Wave Function Collapse (WFC) algorithm, which is most popularly seen in procedural generation in games, to greedily and quickly produce solutions to the MBP that can be further optimized by other metaheuristical approaches. In testing against currently established algorithms, this WFC approach excels in being able to find more optimal solutions to the MBP than other approaches.**

***Index Terms*—Betweeness Problem, Maximum Betweenness Problem, Combinatorial Optimization**

I. Introduction

The Betweeness Problem and its variants are at-least NP-hard problems in computer science and combinatorics. The input is as follows: $S = \{x_1, x_2, \ldots, x_n\}$, consisting of n elements, typically represented as integers for simplicity as well as a set M of constraints in the form of element triples $m = (x_i, x_j, x_k) \in S \times S \times S$. The optimization problem for Betweenness, on which this paper focuses, is concerned with producing an ordering of S, **O** that maximizes the number of constraints in M satisfied. Satisfaction for a constraint requires that x_j, the middle element, is placed between x_i and x_k, or more formally that either $x_i < x_j < x_k$ or $x_k < x_j < x_i$, where $<$ and $>$ compare the order in which elements appear in **O**. To represent satisfaction for a particular solution **O**, an objective function f is defined, such that $f(\mathbf{O}, M)$ represents the number of constraints in M that **O** satisfies, which can also be thought of as a fitness function in evolutionary contexts.

For simplicity, the notation henceforth will see that $S = \{1, 2, \ldots, n\}$, and each constraint $m = (x_i, x_j, x_k) \in S \times S \times S$.

Importantly, the Betweeness Problem has applications in molecular biology, such as placing markers on chromosomes for genome mapping [1].

A. Example Case

For $n = 5$ implying $S = \{1, 2, 3, 4, 5\}$ and M = {(2, 1, 3), (3, 4, 5), (1, 4, 5), (2, 4, 1), (5, 2, 3)}, an optimal ordering of

S can be found by $\mathbf{O} = (3, 1, 4, 2, 5)$, which satisfies all five constraints. For such an $\mathbf{O}$, $f(\mathbf{O}, M) = 5$

II. Existing Literature

In 1979 J. Opatrny proved that the decision variant of the Betweeness Problem is NP-complete using reductions from the 3-satisfiability and the hypergraph 2-colorability problems [2]. In 1998, Chor and Sudan proved that Betweenness is MAX SNP complete and that it is NP-hard to find a solution to the MBP where $\alpha < 47/48$. Here α is defined to be the ratio of the number of satisfied constraints of a particular solution to the number of satisfied constraints in an optimal solution. Chor and Sudan additionally proposed a geometric solution to the problem utilizing semidefinite programming (SDP), where the constraints are converted to quadratic inequalities that are used with SDP to produce a solution. The authors furthermore proved that their approach guaranteed an $\alpha > 1/2$ [3].

In later years, soft computing approaches to the MBP using heuristics and metaheuristics have become popular as practical approximations. In [4] Savić used a Genetic Algorithm metaheuristic to approximate MBP solutions. In general Genetic Algorithms mimic real evolutionary processes like crossover and mutation of genes to introduce change in an existing population of solutions, which are usually randomly generated or generated by other heuristics [5]. Later, in [6] Savić, Kratica, and Fijuljanin improved upon the Genetic Algorithm by using it in conjunction with a Local Search (LS) heuristic, which is technique used to explore the neighborhoods of solutions to find locally optimal solutions. LS is very popular in approximating NP-hard problems in conjunction with metaheuristics, such as in [7] where it is used in the context of the famous NP-hard Traveling Salesman problem.

Later in [8], Filipović, Kartelj, and Matić used an Electromagnetic (EM) metaheuristic, which draws inspiration from Coulomb's Law in physics by seeing a population of solutions as points in a space being drawn together and repelled away according to their fitness. Their solution also utilized a form of Local Search to work with their electromagnetic metaheuristic. When tested on the same MPB problem instances as [4] and [6], the EM approach was found to achieve more optimal solutions than the other two The depth of these algorithms cannot be fully explored in this paper, so please see the original works in [4], [6] and [8] respectively.

III. Algorithm

This paper proposes the WFC-SA-GA algorithm, a Wave Function Collapse (WFC) based heuristic algorithm that works in conjunction with a modifed Simulated Annealing (SA) metaheuristic, as well as a Genetic Algorithm (GA). Although the WFC algorithm is used largely in video game design as a procedural generation tool [9], its constraint-satisfying abilities can be applied to the class of NP-hard problems in the form of fast heuristic algorithm, such as in a solution to the closest string problem in [10].

A. WFC Heuristic Algorithm for Betweenness

The general WFC algorithm follows three main steps: Observe, Collapse, and Propagate. Observe scans the current state and identifies the most constrained element. It then Collapses this constrained element into reality, greedily assigning it to a space in the solution. The propagation step then takes care of the elements most closely tied to the collapsed element, the implementation of which usually differs from problem to problem.

In the case of the optimization problem for Betweenness, the proposed algorithm operates on an ordered sequence of unordered sets $\mathbf{V} = (T_1, T_2, \ldots, T_k)$, where $T_i \subseteq U$ and k can vary. Keeping with the theme of physics-inspired nomenclature, this sequence can be thought of as a solution

state in superposition that itself has not been collapsed into the final ordering $\mathbf{O}$. In contexts surrounding pseudocode and implementation, this paper will refer to these sequences and orderings as vectors due to the way they are implemented in code.

$\mathbf{V}$ is initialized with one empty set, $\mathbf{V} = [\varnothing]$. The current state of m is then analyzed in the Observe step, where a constraint $C = (c_i, c_j, c_k)$ is identified as the most constrained, where the most constrained constraint has a middle element c_j such that among all x_j in constraints in M, c_j is the mode, with ties between constraints broken arbitrarily. After being identified C is then set to be collapsed into the solution space, according to a Decision Model guided by two key paradigms:

- **Paradigm 1:** To collapse a constraint C: For each c_i, c_j, c_k that is not already part of the solution space $\mathbf{V}$, the elements are to be placed in a set of $\mathbf{V}$ such that c_i, c_j, c_k are as spread out in the solution space as possible.
- **Paradigm 2:** In the case that any c_i, c_j, or c_k is in the same set in $\mathbf{V}$ as any other c_i, c_j, or c_k, that set shall be split such that no elements of C share the same set. Furthermore, the resulting split shall keep the indices of the affected elements as close together as possible, in order to maintain the existing ordering in $\mathbf{V}$. This split should also maintain the ordering of constraint C, if possible.

Paradigm 2 implies that the relative positions of sets in $\mathbf{V}$ will remain unchanged with respect to other existing sets. In general, Paradigm 1 is ensured first, which may result in a situation where Paradigm 2 is needed.

In theory, the collapse step of the algorithm collapses specifically the middle element c_j first. Following this decision, it is needed to Propagate the effects of this collapse, which in this algorithm means collapsing the locations of the outer bounds, c_i and c_k, according to the constraint set by the ordering of C.

However in the implementation of this algorithm, the nature of the decision model makes it more efficient and readable to propagate the outer bounds c_i and c_k first and collapse the inner element c_j second, though in practice the order that the elements are collapsed in does not matter.

Once the collapse and propagation for C is complete, C is then removed from m. Barring any influence from metaheuristics, unless the elements of C are already placed in disjoint sets out of order with respect to C, C is guaranteed to be satisfied in the final ordering derived from $\mathbf{V}$. This very strict satisfaction of constraints chosen by the Observe step means that the WFC algorithm for the MBP is very greedy, strongly satisfying the first constraints it attempts to satisfy.

This process of observation, Collapse, and propagation repeats until there are no more constraints in M. When M is empty, $\mathbf{V}$ is collapsed into a solution to the MBP by flattening it into the ordering $\mathbf{O}$. If any elements in S have not been added into $\mathbf{O}$, they are inserted arbitrarily, which is done by appending to the end in this implementation. For an example of the WFC algorithm, please see Table I, which includes each step of the algorithm working.

A generalized WFC implementation is detailed below in Algorithm 1, without any metaheuristics. The Observe, Collapse, and Propagate functions are defined in Algorithm 2, 3, and 4.

The term $placed$ is used in the pseudocode for brevity. An element x for x $\in \{1, 2, 3..., n\}$ is defined to be $placed$ if x can be found in flattenned $\mathbf{V}$.

A Github repository is provided at the end of this paper. The repository contains the source code, implemented in julia 1.10.4, as well as test cases used.

For logic pertaining to the Collapse function in Algorithm 3 or the

Algorithm 1 WFC

Input: integer n, set of constraints M
Output: vector $\mathbf{O}$
$\mathbf{V} \leftarrow [\varnothing]$
while M is not empty **do**
    $C \leftarrow$ Observe(M) ▷ Select a constraint from M
    Collapse($\mathbf{V}$, C) ▷ Apply the constraint to $\mathbf{V}$
    Propagate($\mathbf{V}$, C) ▷ Propagate Changes
    remove C from M
end while
$\mathbf{O} \leftarrow$ flattening of $\mathbf{V}$
add to $\mathbf{O}$ any numbers in $\{1, 2, 3..., n\}$ that are not already in $\mathbf{O}$
return O

Algorithm 2 Observe

Input: set of constraints M
Output: the most constrained[1] constraint C
$M_{max} \leftarrow$ the set of triples that share the highest constriction[1]
$C \leftarrow$ a random constraint in M_{max}
return C

[1]The most constrained triple has a middle element c_j such that among all x_j in triples in M, c_j is the mode.

Algorithm 3 Collapse

Input: vector of sets $\mathbf{V}$, constraint C
if $C.i \in$ and $C.k$ are both placed **then**
    return
else if $C.i \in$ and $C.k$ are both not placed **then**
    place $C.i$ in the first set in $\mathbf{V}$
    place $C.k$ in the last set in $\mathbf{V}$
else ▷ Implied that either $C.i$ or $C.k$ is unplaced
    if $C.i$ is unplaced **then**
        switch values of $C.i$ and $C.k$[1]
    end if
    add $C.k$ to $\mathbf{V}$ to maximize distance from $C.i$
end if
if $C.i$ is after $C.k$ in $\mathbf{V}$ **then**
    switch values of $C.i$ and $C.k$[2]
end if

[1]this is to simplify logic, as then we guarantee that $C.k$ is the outer bound that is unplaced
[2]this is also to simplify logic, as then we guarantee that $C.i$ comes first

Algorithm 4 Propagate

Input: vector of sets $\mathbf{V}$, constraint C
if $C.j$ is unplaced **then**
    if $C.i$ and $C.k$ are in the same set **then**
        $s \leftarrow$ for which $C.i, C.k \in s$
        insert in $\mathbf{V}$ a set s_i right before s
        move $C.i$ to s_i
        insert in $\mathbf{V}$ a set s_k right after s
        move $C.k$ to s_k
    else if $C.k$ is in the set right after that of $C.i$ **then**
        insert in $\mathbf{V}$ a set s right after the set with $C.i$
    else
        $s \leftarrow$ the set in $\mathbf{V}$ in the middle of $C.i$ and $C.k$
    end if
    insert $C.j$ in s
else
    if $C.j$ is after $C.k$ or $C.i$ is after $C.j$ **then**
        return
    else if $C.i, C.j, C.k$ are in the same set **then**
        $s \leftarrow$ for which $C.i, C.j, C.k \in s$
        insert in $\mathbf{V}$ a set s_i right before s
        move $C.i$ to s_i
        insert in $\mathbf{V}$ a set s_k right after s
        move $C.k$ to s_k
    else if $C.k$ and $C.j$ share a set **then**
        insert in $\mathbf{V}$ a set s right after the set with $C.j$
        move $C.k$ to s
    else if $C.i$ and $C.j$ share a set **then**
        insert in $\mathbf{V}$ a set s right after the set with $C.j$
        move $C.j$ to s
    end if
end if

Propagate function in Algorithm 4, an implementation in Julia can be found in the Github provided

B. Metaheuristic 1: Modified Simulated Annealing

The SA metaheuristic is a probabilistic metaheuristic that minimizes some energy function E while allowing for random changes to the heuristic algorithm's choices according to a probability function $P(T)$ based on an exponentially decreasing temperature T [11].

For the MBP and the WFC heuristic, the motivation behind the minimizing of an energy function is already captured through the observing and collapsing of the most common middle element. In essence, the greedy selection of each next triple acts to minimize the energy function.

This application of SA, however, uses a differing approach to the most common probability functions used. The motivation behind using SA is usually to introduce randomness when such a move can lead the algorithm out of local extrema in the possible solution space. Traditionally this follows the curve $e^{-\frac{a}{T}}$, where a is some constant. This graph allows for randomness in the beginning of the search for solutions, but gradually decreases tolerance for randomness with lower temperatures. However for some approaches like WFC for Betweenness, while the changing randomness of SA is very useful,

TABLE I
STEP-BY-STEP EXECUTION OF THE WFC ALGORITHM, USING $n = 5, M = (2,1,3),(3,4,5),(1,4,5),(2,4,1),(5,2,3)$

Step	Action	Selected Constraint	Updated V
1	Observe	(3, 4, 5)	$[\{\}]$
2	Collapse		$[\{3,5\}]$
3	Propagate		$[\{3\},\{4\},\{5\}]$
4	Observe	(1, 4, 5)	$[\{3\},\{4\},\{5\}]$
5	Collapse		$[\{3,1\},\{4\},\{5\}]$
6	Propagate		$[\{3,1\},\{4\},\{5\}]$
7	Observe	(2, 1, 3)	$[\{3,1\},\{4\},\{5\}]$
8	Collapse		$[\{3,1\},\{4\},\{5,2\}]$
9	Propagate		$[\{3\},\{1\},\{4\},\{5,2\}]$
10	Observe	(5, 2, 3)	$[\{3\},\{1\},\{4\},\{5,2\}]$
11	Collapse		$[\{3\},\{1\},\{4\},\{5,2\}]$
12	Propagate		$[\{3\},\{1\},\{4\},\{2\},\{5\}]$
13	Observe	(2, 4, 1)	$[\{3\},\{1\},\{4\},\{2\},\{5\}]$
14	Collapse		$[\{3\},\{1\},\{4\},\{2\},\{5\}]$
15	Propagate		$[\{3\},\{1\},\{4\},\{2\},\{5\}]$

Fig. 1. The logistic function used in the Simulated Annealing

the traditional $e^{-\frac{a}{T}}$ may not be the most appropriate.

This paper's approach to SA is to use a particular curve to create a varied yet strong population of solutions that can be built upon efficiently by the Evolutionary Algorithm without wasting time on bad solutions. For this purpose, this implementation of Simulated Annealing uses the following logistic function that is also pictured in Fig. 1:

$$P(T) = \frac{1}{1 + e^{0.2(T-70)}} \tag{1}$$

This function constricts to randomness to gain very strong choices in the beginning, but allows for more randomness as the temperature goes down from 100 to 0. It should be noted that for this implementation of Simulated Annealing, the temperatures T decrease linearly from 100 to 0 as more triples are attempted to be placed. At any given step, the temperature T is given by Equation 1 below:

$$T = \frac{size(M)}{number\ of\ constraints} \times 100.0 \tag{2}$$

where m is the amount of total constraints, and M is the current set of constraints that have not yet been attempted to be placed.

C. Metaheuristic 2: Genetic Algorithm

A hallmark of the WFC heuristic for Betweenness is its strict commitment to greedily satisfying the first constraints chosen, leading to later choices of constraints to be very likely to fail. This results in an effect where, for a large portion of its runtime, WFC is unable to commit any changes to $\mathbf{V}$ because doing so would contradict with previous changes, particularly in the latter stages of Observe, Collapse, and Propagate.

The solution used by this algorithm is to use a Genetic Algorithm (GA), which is able to introduce randomness to optimize an existing population of solutions over multiple generations of changes [5]. One aspect of this paper's GA is to simply test 1-swaps on particular solutions on the population, replacing current solutions with fitter ones generation to generation. The other aspect of the GA used with WFC is to introduce crossover between strong members of the population. In each generation, the two strongest solutions are used to create a new child solution by copying a random contiguous segment of one parent and filling in unplaced indices using the elements in the other parent. At the end of each generation, all solutions with fitness values below the mean fitness are removed. However, due to the large amount of iterations as well as a motivation to avoid local extrema, those unfit solutions are given a chance to survive each iteration with the hope that further mutations may bring them closer to a more optimal solution.

This approach was largely inspired by the success of using GA on the same problem by [4] and [6].

D. Pseudocode with Metaheuristics

The only function that is changed by the metaheursitcs is the Observe function, called ObserveSA where Simulated Annealing guides the choosing of triples to satisfy. This new Observe function is used by the WFC-SA function, which simply replaces the function call to Observe with a call to ObserveSA.

Algorithm 5 ObserveSA

Input: set of constraints M
Output: the constraint to be satisfied
$T \leftarrow$ a Temperature given by Equation 2
$M_{max} \leftarrow$ the set of triples that share the highest constriction
if random() $< P(T)$ **then** ▷ $P(T)$ is Equation 1
    C is a random triple in M
else
    C is a random triple in M_{max}
end if
return C

Algorithm 6 WFC-SA

Input: integer n, set of constraints M
Output: vector $\mathbf{O}$
$\mathbf{V} \leftarrow [\varnothing]$
while M is not empty **do**
    $C \leftarrow$ ObserveSA(M) ▷ Select a constraint from M
    Collapse($\mathbf{V}$, C) ▷ Apply the constraint to $\mathbf{V}$
    Propagate($\mathbf{V}$, C) ▷ Propagate Changes
    remove C from M
end while
$\mathbf{O} \leftarrow$ flattening of $\mathbf{V}$
add to $\mathbf{O}$ any numbers in $\{1, 2, 3..., n\}$ that are not already in $\mathbf{O}$
return $\mathbf{O}$

The Genetic Algorithm is handled through the final WFC-SA-GA function that uses the WFC-SA function to generate a population $\mathbf{P}$ of solutions. The GA uses multiple constants for determining amount of iterations. In the pseudocode, population size is given by p, the amount of generations is given by g, and the number of attempted 1-swaps per generation is given by h. Additionally the probability that an unfit solution survives a generation is given by r_s. The probability that for each generation, a crossover child is created using the strongest 2 parents is given by r_c.

It should also be mentioned that the fitness function $f(\mathbf{O}, M)$ is used in WFC-SA-GA to compare solutions.

IV. Results

Local testing on the MBP problem was performed on a 13th GEn Intel(R) Core(TM) i7-13700HX laptop CPU at base speed of 2.10 GHz and 36 GB 4800 MT/s RAM on Windows 11 Home Version 23H2.

Testing was performed on a set of 22 problem instances with varying problem dimensions. These cases were used by [4], [6], and [8] for their own testing.

Algorithm 7 WFC-SA-GA

Input: integer n, set of constraints M
Output: vector $\mathbf{O}$
$\mathbf{P} \leftarrow []$ ▷ the initial population of solutions
while size($\mathbf{P}$) is not p **do** ▷ p is initial population size
    add the result of WFC-SA(n, M) to $\mathbf{P}$
end while
for $i = 1$ **to** g **do** ▷ g is number of generations
    for $j = 1$ **to** size($\mathbf{P}$) **do**
        $\mathbf{v} \leftarrow []$
        for $k = 1$ **to** h **do** ▷ h is number of 1-swaps
            $s \leftarrow$ copy of $\mathbf{P}[j]$
            perform 1-swap on two random indices of s
            if $f(s, M) > f(\mathbf{P}[j], M)$ **then**
                add s to $\mathbf{v}$
            end if
        end for
        if $\mathbf{v}$ is not empty **then**
            replace $\mathbf{P}[j]$ with the most fit solution in $\mathbf{v}$
        end if
    end for
    for $j = 1$ **to** size($\mathbf{P}$) **do**
        if $f(\mathbf{P}[j], M) <$ mean fitness in $\mathbf{P}$ **then**
            if random() $< r_s$ **then**
                remove $\mathbf{P}[j]$ from $\mathbf{P}$
            end if
        end if
    end for
    if random() $< r_c$ **then** ▷ performing crossover
        $s_1 \leftarrow$ the most fit solution in $\mathbf{P}$
        $s_2 \leftarrow$ the second most fit solution in $\mathbf{P}$
        $s_{new} \leftarrow []$
        push a random subsequence in s_1 to s_{new}
        push values from s_2 to the end of s_{new}, skipping duplicates
    end if
end for

Performance testing was performed on the WFC-SA-GA algorithm and the EM algorithm in Table II. It should be noted that the WFC-SA-GA algorithm was implemented in julia 1.10.4, while the EM algorithm was implemented in the C programming language and compiled in Visual Studio 2010 [8]. Therefore, while algorithm runtimes are provided, they serve more as context rather than a point of comparison between the algorithms. Additionally, in the original EM paper [8] performance was averaged over 20 trials, with the best solutions being recorded over those trials. In this paper's testing, a similar approach was attempted, using 20 trials of both the EM and WFC-SA-GA algorithms. In Table II $instance$ refers to the MBP problem instance, where 10-20 refers to a problem with an ordering

of 10 elements and a set of 20 constraints. Opt refers to the optimal solution to a given case, if such a solution is known. $WFC(best)$ refers to the fitness of the best solution achieved by WFC-SA-GA in 20 trials, and $WFC(avg)$ refers to the average of the solutions generated. Similarly $EM(best)$ refers to the best of the 20 EM trials, and $EM(avg)$ refers to their mean. Columns 5 and 6(t_{avg} and t_{tot}) indicate the average running time for each trial of WFC-SA-GA and the total runtime of the 20 trials respectively. Similarly columns 10 and 11 say the same for the EM algorithm.

This paper's EM testing resulted in very similar results to those published in [8], with a few small differences. Therefore the original results of the EM paper are included in the table, labeled $EM(og)$. Only the fitnesses of the original EM solutions are included.

Additionally, the work of the WFC-SA-GA algorithm is broken down in Table IV. Similarly to Table II, the columns with SA and SA-GA represent the two parts of the entire WFC-SA-GA algorithm. SA refers to the generation of population solutions, which is assisted by Simulated Annealing. SA-GA refers to the entire algorithm, including both generation of a population and the Genetic Algorithm that is applied afterwards. The results under SA-GA are the same as WFC in Table II.

Table IV provides a focused comparison on only the maximum fitnesses achieved by each algorithm. Included are the results from WFC-SA-GA, EM, and the Genetic Algorithm + Local Search algorithm in [6]. WFC refers to the best solutions of the WFC-SA-GA algorithm found in testing, EM refers to the best solutions found in both this paper's testing and those reported in [8], and $GA + LS$ refers to the maximum fitness reported in the testing of [6].

TABLE III
COMPARISON OF BEST SOLUTIONS ACHIEVED AMONG WFC-SA-GA, EM, AND GA-LS

Instance	Opt	WFC	EM	GA+LS	Best
10-20	16	16	16	16	-
10-50	29	29	29	29	-
10-100	50	50	50	50	-
11-20	14	14	14	14	-
11-50	33	33	33	33	-
11-100	55	55	55	55	-
12-20	17	17	17	17	-
12-50	34	34	34	33	WFC, EM
12-100	56	56	56	56	-
15-30	26	26	26	25	WFC, EM
15-70	-	46	46	46	-
15-200	-	106	106	105	WFC, EM
20-40	37	37	37	37	WFC, EM
20-100	-	68	68	66	WFC, EM
20-200	-	117	117	114	WFC, EM
30-60	55	55	54	53	WFC
30-150	-	109	111	111	EM, GA+LS
30-300	-	183	185	179	EM
50-100	-	90	87	86	WFC
50-200	-	157	153	151	WFC
50-400	-	269	259	265	WFC
50-1000	-	539	536	532	WFC

TABLE II
PERFORMANCE COMPARISON OF WFC-SA-GA AND EM ALGORITHMS

Instance	Opt	WFC(Best)	WFC(avg)	$t_{avg}(s)$	$t_{tot}(s)$	EM(og)	EM(best)	EM(avg)	$t_{avg}(s)$	$t_{tot}(s)$
10-20	16	16	16	0.0354	0.7085	16	16	16	0.0417	0.833
10-50	29	29	29	0.0680	1.3605	29	29	29	0.1009	2.018
10-100	50	50	49.9	0.1453	2.9065	50	50	50	0.2145	4.289
11-20	14	14	14	0.0631	1.2616	14	14	14	0.0413	0.826
11-50	33	33	32.7	0.0886	1.7712	33	33	33	0.1327	2.653
11-100	55	55	55	0.1638	3.2752	55	55	55	0.2563	5.125
12-20	17	17	17	0.0602	1.2049	17	17	17	0.0522	1.043
12-50	34	34	34	0.1410	2.8199	34	34	34	0.1432	2.864
12-100	56	56	56	0.2342	4.6835	56	56	56	0.3041	6.081
15-30	26	26	24.9	0.1483	2.9659	26	26	25.2	0.1126	2.252
15-70	-	46	45.9	0.2276	4.5516	46	46	46	0.2730	5.459
15-200	-	106	105.7	0.7314	14.6284	106	106	106	1.1234	22.468
20-40	37	37	36.6	0.2858	5.7151	37	37	36.3	0.2728	5.455
20-100	-	68	67.2	0.7367	14.7337	68	68	66.9	0.7790	15.580
20-200	-	117	115.7	1.4107	28.2132	117	117	116.2	2.1572	43.144
30-60	55	55	54.1	1.1439	22.8772	54	54	53.2	0.6869	13.738
30-150	-	109	105.8	2.6648	53.2958	111	110	106.7	2.7595	55.190
30-300	-	183	179.8	5.7167	114.3331	185	185	179.8	6.8974	137.948
50-100	-	90	86.9	2.3416	46.8318	87	87	85.7	0.6481	12.961
50-200	-	157	150.3	4.9299	98.5976	153	153	147.8	2.0413	40.825
50-400	-	269	261.5	33.3075	357.8859	259	256	252.2	5.2475	104.950
50-1000	-	539	527.1	115.914	2318.279	536	536	524.6	58.214	1164.28

TABLE IV
PERFORMANCE COMPARISON OF WFC-SA-GA AND WFC-SA ALGORITHMS

Instance	Opt	SA(Best)	SA(avg)	$t_{avg}(s)$	$t_{tot}(s)$	SA-GA(Best)	SA-GA(avg)	$t_{avg}(s)$	$t_{tot}(s)$
10-20	16	16	15.85	0.0144	0.2878	16	16	0.0354	0.7085
10-50	29	29	27.75	0.0340	0.6802	29	29	0.0680	1.3605
10-100	50	49	47.7	0.0752	1.5047	50	49.9	0.1453	2.9065
11-20	14	14	14	0.0133	0.2669	14	14	0.0631	1.2616
11-50	33	33	31	0.0353	0.7069	33	32.7	0.0886	1.7712
11-100	55	54	50.8	0.0854	1.7076	55	55	0.1638	3.2752
12-20	17	17	16.25	0.0136	0.2725	17	17	0.0602	1.2049
12-50	34	34	31.45	0.0367	0.7347	34	34	0.1410	2.8199
12-100	56	54	50.7	0.0808	1.6154	56	56	0.2342	4.6835
15-30	26	25	23	0.0212	0.4247	26	24.85	0.1483	2.9659
15-70	-	44	41.8	0.0543	1.0855	46	45.9	0.2276	4.5516
15-200	-	98	91.25	0.2147	4.2935	106	105.7	0.7314	14.6284
20-40	37	33	30.75	0.0295	0.5895	37	36.55	0.2858	5.7151
20-100	-	60	56.65	0.0947	1.8934	68	67.15	0.7367	14.7337
20-200	-	103	97.2	0.2157	4.3134	117	115.7	1.4107	28.2132
30-60	55	50	45.5	0.0477	0.9550	55	54.1	1.1439	22.8772
30-150	-	88	83.1	0.1631	3.2628	109	105.8	2.6648	53.2958
30-300	-	151	142.55	0.3938	7.8765	183	179.8	5.7167	114.3331
50-100	-	74	71.8	0.0982	1.9645	90	86.9	2.3416	46.8318
50-200	-	119	113.75	0.2610	5.2202	157	150.3	4.9299	98.5976
50-400	-	204	195.4	0.6819	13.6379	269	261.5	33.3075	357.8859
50-1000	-	420	411.05	3.0143	60.2869	539	527.05	115.9139	2318.2787

V. DISCUSSION

Although a direct comparison between the runtimes of the julia based WFC-SA-GA and the C based EM is not possible, it is clear that for the trials done in Table II, the runtimes of the two algorithms are roughly on the same order of magnitude. However in general, the speed of the WFC-SA-GA algorithm appears to suffer for larger problems, especially those with 50 elements in an ordering.

In Table IV, the population generation (WFC-SA) and the full population generation plus post-optimization (WFC-SA-GA) algorithms are compared. It is clear that a majority of the runtime of WFC-SA-GA is spent on the Genetic Algorithm post-optimization, while the generation of a population takes very little time. despite its speed, the population generation (WFC-SA) algorithm does suffer in accuracy for larger problems. Depending on use cases, though, a high speed greedy solution that produces lower quality solutions may have its uses, though that is out of the scope of this paper.

When comparing the average fitness of EM and WFC-SA-GA in Table II, out of the 14 instances that were not ties, both algorithms each had 7 instances where they beat the other. The instances that EM won in tended to be smaller problems, including 11-50, 15-30, and 15-70, which were especially small compared to other instances. For 10-100, 11-50, and 15-70, 15-200 the average fitness for EM was equal to the maximum known fitness, yet WFC-SA-GA failed to reach this maximum known fitness in some trials, resulting in an average that is slightly lower than the known best solutions. Even though for those cases WFC-SA-GA was likely to be less consistent, it did however reach the best known solution in the allocated 20 trials. On the other hand, the instances that WFC-SA-GA won in tended to appear for generally larger problem sizes, such as all four of the instances with 50 element orderings.

In Table III the best solutions found by the WFC-SA-GA, EM and GA+LS algorithms are compared, and there was only one case (30-150) out of the 13 cases that were not tied, where GA+LS outperformed WFC-SA-GA. Between WFC-SA-GA and EM, WFC-SA-GA reached a fitter solution in five out of seven instances that were not ties. The experiments in this paper using WFC-SA-GA were able to discover new best solutions for five of the problem instances, and in a reasonable amount of time, as seen in Table II.

In general it can be said that despite perhaps not excelling in terms of speed, WFC-SA-GA is a reasonably fast approach. It is

furthermore stronger than EM or GA+LS in terms of finding a solution that maximizes the number of satisfied constraints.

VI. Conclusions

This paper has introduced a new method for approximating the MBP, called WFC-SA-GA. It uses a modified Simulated Annealing metaheuristic alongside a heuristic largely inspired by the Wave Function Collapse algorithm to quickly generate a population of strong but varied solutions. This population is then optimized through the use of a Genetic Algorithm metaheuristic.

Experimental results show WFC-SA-GA being very capable in reaching new best known solutions for multiple well documented test cases used in [4], [6], and [8], more often than not surpassing the Electromagnetic metaheuristic algorithm and Genetic Algorithm + Local Search. Due to differences in testing conditions between the julia based WFC-SA-GA and the C based Electromagnetic metaheuristic algorithm, this paper is not able to confidently compare the speed of the two algorithms.

The WFC-SA-GA algorithm was used in this paper for solely optimizing the maximum fitness of any given MBP instance, but aspects of the WFC algorithm may prove useful in tackling more niche versions of the Betweeness Problem. For example, in WFC-SA-GA, the contraints chosen to be satisfied by the WFC heuristic were automatically chosen based on a perceived level of constriction, which was decided to be the frequency of the middle element. However for versions of the problem where certain constraints may be prioritized, such as a problem with weighted constraints, the speed and greediness of WFC may be very useful.

A Github link is provided containing relevant material.

Github: https://github.com/DanielQ-51/WFC-SA-GA.git

VII. Acknowledgements

The authors would like to thank Professor Aleksandar Kartelj, an author of [8], for providing resources from their previous work, which was instrumental in the comparison and analysis performed in this paper.

References

[1] D. Slonim, L. Kruglyak, L. Stein, and E. Lander, "Building human genome maps with radiation hybrids," in *RECOMB97: The First Annual Conference on Research in Computational Molecular Biology*, 1997, pp. 277–286.

[2] J. Opatrny, "Total ordering problem," *SIAM Journal on Computing*, vol. 8, no. 1, pp. 111–114, 1979.

[3] B. Chor and M. Sudan, "A geometric approach to betweenness," *SIAM Journal on Discrete Mathematics*, vol. 11, no. 4, pp. 511–523, 1998.

[4] A. Savić, "On solving the maximum betweenness problem using genetic algorithms," *Serdica Journal of Computing*, vol. 3, pp. 299–308, 2009.

[5] S. Mirjalili and S. Mirjalili, "Genetic algorithm," *Evolutionary algorithms and neural networks: theory and applications*, pp. 43–55, 2019.

[6] A. Savić, J. Kratica, and J. Fijuljanin, "Hybrid genetic algorithm for solving of maximum betweenness problem," in *Proceedings of the 1st International and 10thBalkan Conference on Operational Research-BALCOR*, 2011, pp. 402–408.

[7] C. Rego and F. Glover, "Local search and metaheuristics," *The traveling salesman problem and its variations*, pp. 309–368, 2007.

[8] V. Filiović, A. Kartelj, and D. Matić, "An electromagnetism metaheuristic for solving the maximum betweenness problem," *Applied Soft Computing*, vol. 13, no. 2, pp. 1303–1313, 2013.

[9] H. Kim, S. Lee, H. Lee, T. Hahn, and S. Kang, "Automatic generation of game content using a graph-based wave function collapse algorithm," in *2019 IEEE conference on games (CoG)*. IEEE, 2019, pp. 1–4.

[10] S. Xu and D. Perkins, "A heuristic solution to the closest string problem using wave function collapse techniques," *IEEE Access*, vol. 10, pp. 115 869–115 883, 2022.

[11] D. Bertsimas and J. Tsitsiklis, "Simulated annealing," *Statistical science*, vol. 8, no. 1, pp. 10–15, 1993.